

Add One to a Linked List

My DSA Study Notes

1. Introduction

We are given a number where every digit is stored inside one node of a singly linked list. We have to add 1 to that number and return the head of the resulting linked list.

The main difficulty is that addition starts from the last digit, but a singly linked list only moves forward.

2. Problem Statement

Each node represents one digit. Add 1 to the complete number and return the new head.

Example:

1 -> 5 -> 9 -> NULL

159 + 1 = 160

Result:

1 -> 6 -> 0 -> NULL

Another example:

9 -> 9 -> 9 -> NULL

999 + 1 = 1000

Result:

1 -> 0 -> 0 -> 0 -> NULL

3. Required Concepts

- Singly linked-list traversal
- Reversing a linked list
- Carry during addition
- Recursion and backtracking
- Handling an extra digit when the final carry is 1

4. Why Do We Need a Special Approach?

For normal addition, I start from the last digit.

1 -> 5 -> 9

Addition starts here
↓
[9]

But in a singly linked list, I cannot move from 9 back to 5 and then to 1. There is no previous pointer.

So there are two useful ways to solve it:

1. Reverse the list, perform the addition, then reverse it back.
2. Use recursion so that backtracking gives me the same right-to-left processing order.

5. Approach 1 — Reverse, Add, Reverse

First reverse the list. Now the last digit becomes the first node, so I can process the digits from right to left.

Original:

1 -> 5 -> 9 -> NULL

Reverse:

9 -> 5 -> 1 -> NULL

```

Add 1:
9 + 1 = 10
digit = 0
carry = 1

5 + 1 = 6
digit = 6
carry = 0

Stop.

Current:
0 -> 6 -> 1 -> NULL

Reverse again:
1 -> 6 -> 0 -> NULL

```

If the carry is still 1 after the last node, I need one extra node at the front.

```

999

Reverse:
9 -> 9 -> 9

After addition:
0 -> 0 -> 0
carry = 1

Create:
1 -> 0 -> 0 -> 0

```

6. Approach 1 — Pseudocode

```

addOne(head):

    head = reverse(head)

    temp = head
    carry = 1

    while temp != null:

        temp.val = temp.val + carry

        if temp.val < 10:
            carry = 0
            break

        else:
            temp.val = 0
            carry = 1

        temp = temp.next

    head = reverse(head)

    if carry == 1:
        newNode = new Node(1)
        newNode.next = head
        head = newNode

    return head

```

7. Approach 1 — Core Code

```

head = reverse(head);

ListNode temp = head;
int carry = 1;

while(temp != null){

    temp.val = temp.val + carry;

```

```

        if(temp.val < 10){
            carry = 0;
            break;
        }
        else{
            temp.val = 0;
            carry = 1;
        }

        temp = temp.next;
    }

    head = reverse(head);

    if(carry == 1){
        ListNode newNode = new ListNode(1);
        newNode.next = head;
        head = newNode;
    }

    return head;
}

```

8. Approach 1 — Complete Java Code

```

class Solution {

    private ListNode reverse(ListNode head) {

        ListNode prev = null;
        ListNode temp = head;

        while(temp != null){

            ListNode front = temp.next; // Save next node
            temp.next = prev;           // Reverse the link
            prev = temp;                 // Move prev forward
            temp = front;                // Move temp forward
        }

        return prev;                    // New head
    }

    public ListNode addOne(ListNode head) {

        head = reverse(head);

        ListNode temp = head;
        int carry = 1;

        while(temp != null){

            temp.val = temp.val + carry;

            if(temp.val < 10){
                carry = 0;
                break;
            }
            else{
                temp.val = 0;
                carry = 1;
            }

            temp = temp.next;
        }

        head = reverse(head);

        if(carry == 1){
            ListNode newNode = new ListNode(1);
            newNode.next = head;
            head = newNode;
        }

        return head;
    }
}

```

```
}  
}
```

9. Approach 1 — Static Visual

ORIGINAL

```
HEAD  
|  
v  
[1] -> [5] -> [9] -> NULL
```

REVERSE

```
HEAD  
|  
v  
[9] -> [5] -> [1] -> NULL
```

ADD

```
[0] -> [6] -> [1] -> NULL  
^  
carry becomes 0
```

REVERSE AGAIN

```
HEAD  
|  
v  
[1] -> [6] -> [0] -> NULL
```

10. Approach 1 — Complexity

Time: $O(n)$. I reverse the list, traverse it once for addition, and reverse it again. These are all linear passes.

Auxiliary Space: $O(1)$. I only use pointers and the carry variable.

11. Approach 2 — Recursion + Backtracking

Instead of physically reversing the list, I can use recursion to reach the last node first. The addition is performed while the recursive calls return.

```
1 -> 5 -> 9 -> NULL
```

```
helper(1)  
|  
helper(5)  
|  
helper(9)  
|  
helper(null)
```

At null:
carry = 1

Backtracking:

```
9 + 1 = 10  
9 becomes 0  
carry = 1
```

```
5 + 1 = 6  
5 becomes 6  
carry = 0
```

```
1 + 0 = 1  
1 stays 1
```

```
carry = 0
```

Result:

```
1 -> 6 -> 0 -> NULL
```

12. Approach 2 — Pseudocode

```
addOne(head):
```

```
    carry = helper(head)
```

```
    if carry == 1:
```

```
        newNode = new Node(1)
```

```
        newNode.next = head
```

```
        head = newNode
```

```
    return head
```

```
helper(temp):
```

```
    if temp == null:
```

```
        return 1
```

```
    carry = helper(temp.next)
```

```
    temp.val = temp.val + carry
```

```
    if temp.val < 10:
```

```
        return 0
```

```
    temp.val = 0
```

```
    return 1
```

13. Approach 2 — Core Code

```
if(temp == null){
```

```
    return 1;
```

```
}
```

```
int carry = helper(temp.next);
```

```
temp.val = temp.val + carry;
```

```
if(temp.val < 10){
```

```
    return 0;
```

```
}
```

```
temp.val = 0;
```

```
return 1;
```

```
int carry = helper(head);
```

```
if(carry == 1){
```

```
    ListNode newNode = new ListNode(1);
```

```
    newNode.next = head;
```

```
    head = newNode;
```

```
}
```

```
return head;
```

14. Approach 2 — Complete Java Code

```
class Solution {
```

```
    private int helper(ListNode temp) {
```

```
        // When we reach the end, +1 becomes the initial carry.
```

```
        if(temp == null){
```

```
            return 1;
```

```
        }
```

```

        // Go to the last digit first.
        int carry = helper(temp.next);

        // Add the carry while coming back.
        temp.val = temp.val + carry;

        // No carry needs to move further.
        if(temp.val < 10){
            return 0;
        }

        // 10 becomes 0 and sends carry 1 backward.
        temp.val = 0;
        return 1;
    }

    public ListNode addOne(ListNode head) {

        int carry = helper(head);

        // A remaining carry creates a new first digit.
        if(carry == 1){
            ListNode newNode = new ListNode(1);
            newNode.next = head;
            head = newNode;
        }

        return head;
    }
}

```

15. Approach 2 — Static Visual

CALLING

```

[1] -> [5] -> [9] -> NULL
 |      |      |
 v      v      v
helper helper helper
              |
              v
              NULL

```

BACKTRACKING

```

[9] + 1 = 10
[9] -> 0
carry = 1

```

```

[5] + 1 = 6
[5] -> 6
carry = 0

```

```

[1] + 0 = 1
[1] -> 1
carry = 0

```

RESULT

```

[1] -> [6] -> [0] -> NULL

```

ALL 9s

```

[9] -> [9] -> [9] -> NULL

```

Backtracking:

```

[9] -> [0]
[9] -> [0]
[9] -> [0]

```

Final carry = 1

Create new node:

[1] -> [0] -> [0] -> [0] -> NULL

16. Approach 2 — Complexity

Time: $O(n)$. Every node is processed once during the recursive backtracking.

Auxiliary Space: $O(n)$. The recursive calls occupy the call stack.

17. Edge Cases

Empty list: follow the platform's constraints. If null is allowed, handle it according to the expected problem definition.

One node, value less than 9: 5 becomes 6.

One node, value 9: 9 becomes 10, so the answer is 1 -> 0.

Carry stops early: 159 becomes 160; once the carry becomes 0, earlier digits do not change.

All digits are 9: 999 becomes 1000, so a new head is required.

18. Complexity Comparison

Approach	Time	Auxiliary Space	Main Idea
Reverse + Add + Reverse	$O(n)$	$O(1)$	Reverse to process from the last digit
Recursion + Backtracking	$O(n)$	$O(n)$	Reach the end, then add while returning

19. Quick Revision

- Addition starts from the last digit.
- Singly linked lists cannot move backward directly.
- Iterative idea: reverse $\rightarrow$ add $\rightarrow$ reverse.
- Recursive idea: go to the end $\rightarrow$ add during backtracking.
- Carry starts as 1 because the required operation is +1.
- If the final carry is 1, create a new node with value 1.
- Iterative auxiliary space = $O(1)$.
- Recursive auxiliary space = $O(n)$.

20. Conclusion

The key idea is to process the digits from right to left. Since a singly linked list does not have a previous pointer, I can either reverse the list or use recursion and backtracking. Both approaches take $O(n)$ time, while the iterative approach uses $O(1)$ auxiliary space.

My main takeaway: when the required operation needs backward movement in a singly linked list, think about reversal or recursion/backtracking.